Apuntes · Clase 34 · Árboles de decisión

Para leer antes de los ejercicios · lectura de ~50 min

Lo más importante, en 6 ideas

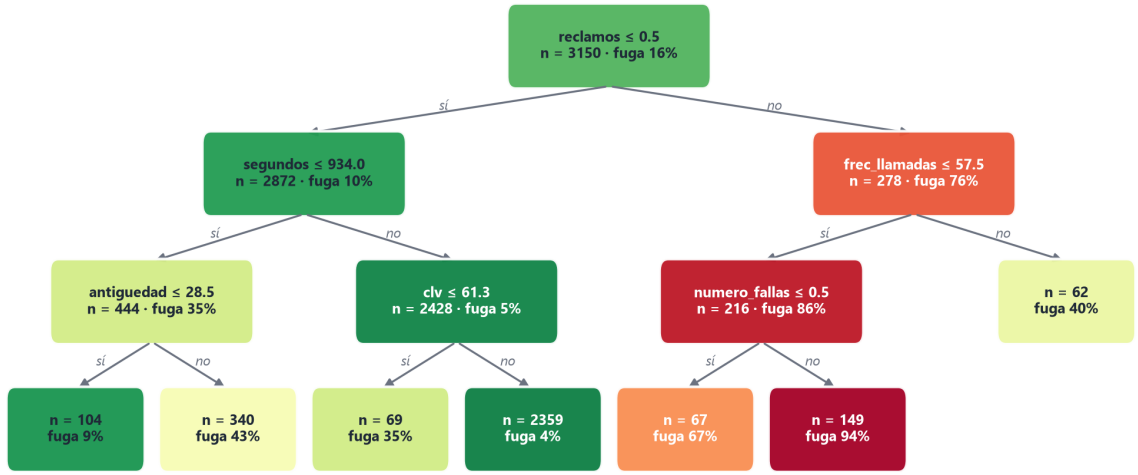
1. **Rectángulos.** El árbol corta el espacio con preguntas sí/no; cada hoja predice la tasa de fuga de su grupo.
2. **Impureza.** Gini o entropía miden la mezcla de un nodo. El mejor corte es el de mayor ganancia.
3. **Voraz.** Elige lo mejor en cada nodo, sin mirar adelante.
4. **Sobreajuste.** Sin límite, memoriza. Se controla con pre-poda (límites) y post-poda (ccp_alpha validado).
5. **Importancia.** Usa permutación sobre el test; la impureza premia a las variables con muchos valores.
6. **Al bosque.** Un árbol es explicable pero inestable; promediar muchos (Random Forest) predice mucho mejor.

Cada sección trae una explicación, un **ejemplo resuelto con números distintos** a los de los ejercicios y una **pista** para el notebook `C34_ArbolesDecision.ipynb` o el cuaderno `C34_Ejercicios_Cuaderno_ArbolesDecision.pdf`.

1 Qué hace un árbol

Un árbol de decisión hace preguntas encadenadas: «¿reclamó?», «¿usa menos de 934 segundos?»... Cada respuesta te lleva por una rama hasta una **hoja**. La predicción es la proporción de fuga de los clientes de entrenamiento que llegaron a esa hoja. A diferencia de la logística, no hay una fórmula con coeficientes: hay **regiones**.

Árbol de profundidad 3 (min_samples_leaf = 60) con los datos de la clase. Color: rojo = más fuga.



Ejemplo resuelto · recorrer el árbol con un cliente

Cliente: reclamó, hizo 40 llamadas, tuvo 0 fallas.

Nodo	Pregunta	Respuesta → rama
Raíz	reclamos ≤ 0.5	no (sí reclamó) → derecha
Nivel 2	frec_llamadas ≤ 57.5	sí (40) → izquierda
Nivel 3	numero_fallas ≤ 0.5	sí (0 fallas) → izquierda
Hoja	67 clientes	probabilidad de fuga 67%

Lift de esa hoja: 0.67 / 0.157 = 4.3 veces la tasa general.

Pista para la Parte A del notebook y el Bloque 1 del cuaderno
En cada nodo, «sí» significa que la condición x ≤ umbral se cumple y vas a la izquierda. Escribe la ruta como regla: «reclamó Y llamadas ≤ 57.5 Y fallas ≤ 0.5».

2 Impureza: Gini y entropía

Un nodo es **puro** si todos sus clientes son de la misma clase. La impureza mide cuánto se aleja de eso. Con dos clases y p = proporción de fuga en el nodo:

$$\text{Gini} = 1 - p^2 - (1-p)^2 \cdot \text{Entropía} = -p\cdot\log_2(p) - (1-p)\cdot\log_2(1-p)$$

Ejemplo resuelto · tres nodos

Nodo	p (fuga)	Gini	Entropía
Puro	0	1 − 0 − 1 = 0	0
20% de fuga	0.2	1 − 0.04 − 0.64 = 0.32	−0.2·(−2.32) − 0.8·(−0.32) = 0.722
Mitad y mitad	0.5	1 − 0.25 − 0.25 = 0.5	1 (el máximo)

Para la entropía necesitas $\log_2$: $\log_2(x) = \ln(x) / \ln(2)$. Por ejemplo, $\log_2(0.2) = -1.609 / 0.693 = -2.32$.

3 Elegir el corte: la ganancia

Para cada variable y cada umbral, el árbol calcula la impureza de los dos hijos, la pondera por su tamaño y la compara con la del padre. La diferencia es la **ganancia**. Gana el corte con mayor ganancia, y el proceso se repite en cada hijo.

Ejemplo resuelto · padre de 100 clientes con 20 fugas (Gini 0.32)

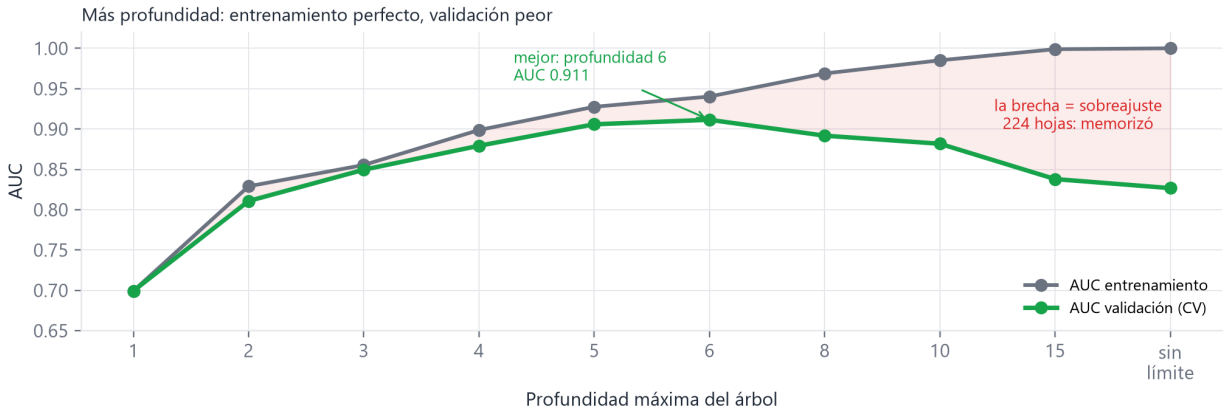
Corte	Hijo izquierdo	Hijo derecho	Gini ponderado	Ganancia
A	40 clientes, 16 fugan: Gini 0.48	60 clientes, 4 fugan: Gini 0.124	0.4·0.48 + 0.6·0.124 = 0.267	0.32 − 0.267 = 0.053
B	50 clientes, 10 fugan: Gini 0.32	50 clientes, 10 fugan: Gini 0.32	0.32	0

El árbol elige A. B produce hijos idénticos al padre: ganancia cero. Es lo que le pasa a **edad** en la clase (ganancia 0.0005).

Pista para la Parte B del notebook y los Bloques 2 a 4 del cuaderno
Ordena el cálculo siempre igual: (1) proporción de cada hijo, (2) Gini de cada hijo, (3) ponderar por n_{hijo}/n, (4) restar al Gini del padre.

4 Sobreajuste y validación cruzada

Si lo dejas crecer sin límite, el árbol sigue partiendo hasta que cada hoja es pura: 224 hojas con los datos de la clase. Acierta todo el entrenamiento (AUC 1.000), pero en datos nuevos cae a 0.827: memorizó el ruido. La **curva de profundidad** compara el AUC de entrenamiento con el de validación cruzada para cada profundidad, y la mejor es la que maximiza el de validación (aquí, 6, con 0.911).



5 Pre-poda y post-poda

La **pre-poda** limita al árbol mientras crece (max_depth, min_samples_leaf…). La **post-poda** lo deja crecer y después recorta las ramas que no compensan su costo. sklearn usa la poda por costo-complejidad: cada hoja «cuesta» α.

Ejemplo resuelto · ¿qué árbol gana según α?

Árbol grande T1: error 0.10 con 20 hojas · árbol chico T2: error 0.13 con 5 hojas. Costo = error + α · hojas.

α	Costo T1	Costo T2	Gana
0.001	0.10 + 0.020 = 0.120	0.13 + 0.005 = 0.135	T1 (el grande)
0.005	0.10 + 0.100 = 0.200	0.13 + 0.025 = 0.155	T2 (el chico)

Empatan cuando 0.10 + 20α = 0.13 + 5α → α = 0.002. Con α mayor, conviene el árbol podado. El α final se elige con validación cruzada, no a ojo.

Pista para la Parte C del notebook y los Bloques 5 y 6 del cuaderno

La pregunta de cada hoja es: ¿reduce el error en más de α? Si no, se recorta.

6 Importancia de variables: impureza vs permutación

La importancia «por impureza» (`feature_importances_`) suma cuánto redujo la impureza cada variable en todos sus cortes. Tiene un sesgo: las variables con muchos valores distintos tienen más umbrales para probar y, por azar, alguno reduce la impureza. La importancia **por permutación** desordena una variable en datos no vistos y mide cuánto cae el AUC: si el modelo no la necesitaba, casi no cae.

Ejemplo resuelto · leer una importancia por permutación

Variable	AUC original	AUC con la variable desordenada	Importancia
reclamos	0.900	0.865	0.035
una variable ruidosa	0.900	0.899	0.001

En la clase, clv (2.654 valores distintos) parecía importante por impureza (0.109) pero no por permutación (0.014). estado (binaria) era baja por impureza (0.045) y casi igual que clv por permutación (0.019).

7 Fortalezas, límites y el bosque

Fortalezas	Límites
Se dibuja y se explica a cualquiera	Inestable: pocas observaciones distintas cambian el árbol
Captura no linealidad e interacciones solas	Solo corta perpendicular a los ejes
No hay que escalar: invariante a log, raíz…	Predice constantes por región: no extrapola
Mezcla variables numéricas y categóricas	Sobreajusta con facilidad
Selecciona variables mientras entrena	Un árbol solo suele rendir menos que una logística

Ejemplo resuelto · por qué promediar árboles reduce el error

Si cada árbol tiene varianza $\sigma^2 = 1$ y la correlación entre árboles es ρ, la varianza del promedio de B árboles es $\rho\sigma^2 + (1 - \rho)\sigma^2/B$.

ρ	B	Varianza del promedio
0.3	10	0.3 + 0.7/10 = 0.37
0.3	100	0.3 + 0.7/100 = 0.307
0	100	1/100 = 0.01

Más árboles ayudan, pero el piso es $\rho\sigma^2$. Por eso el bosque además **decorrelaciona** los árboles: cada corte mira solo $\sqrt{12} \approx 3$ variables al azar. Es el tema de la Clase 35.

8 Chuleta de código

Quiero…	Código
Entrenar un árbol	DecisionTreeClassifier(max_depth=3, min_samples_leaf=60, random_state=0).fit(X, y)
Ver las reglas	export_text(arbol, feature_names=list(X.columns))
Dibujarlo	plot_tree(arbol, feature_names=X.columns, filled=True)
Hoja de cada cliente	arbol.apply(X)
Curva de profundidad	cross_validate(..., return_train_score=True) en un loop de max_depth
Alphas candidatos	arbol.cost_complexity_pruning_path(X, y).ccp_alphas
Elegir α	GridSearchCV(DecisionTreeClassifier(), {'ccp_alpha': alphas}, cv=skf, scoring='roc_auc')
Importancia sesgada	modelo.feature_importances_
Importancia correcta	permutation_importance(modelo, X_test, y_test, n_repeats=20, scoring='roc_auc')
Bosque	RandomForestClassifier(n_estimators=300, max_features='sqrt', n_jobs=-1)

9 Errores comunes

Error	Por qué está mal	Qué hacer
Dejar el árbol sin límite	Memoriza (224 hojas, AUC 0.83)	Curva de profundidad + poda
Hojas de muy pocos clientes	Tasas de ruido	min_samples_leaf con criterio
Elegir α a ojo	Puede sobrepodar o quedarse corto	GridSearchCV con validación cruzada
Confiar en feature_importances_	Premia la cardinalidad	Permutación sobre el test
Escalar variables para un árbol	No cambia nada	Ahorra ese paso
Mirar solo el AUC promedio	Puede esconder mucho desvío	Reportar promedio y desvío
Elegir árbol cuando solo importa predecir	Un bosque predice mucho mejor	Preguntar para qué es el modelo

Lo que debes recordar

- Orden sugerido: lee estos apuntes → Bloques 1–4 del cuaderno → Partes A y B del notebook → Bloques 5–6 → Parte C → Bloques 7–8 → Partes D y E → ejercicio de cierre.
- Si te trabas más de 10 minutos, mira la solución, entiéndela y vuelve a hacerlo sin mirar.